Dell ECS: Deploying the Elastic Stack with Searchable Snapshots and Frozen Tier

Pull ECS Access Logs, through Syslog, into Elastic to search and visualize transactional data

July 2022

H19217

Deployment Guide

Abstract

Optimize your Elastic storage architecture by keeping your most frequently searched Elastic data in the Hot tier and allow Elastic's integrated Lifecycle Management to move the data through the various phases down to the S3 Frozen tier on ECS.

Dell Technologies

Copyright

Contents

Executive summary

Overview

The explosive growth of unstructured data and cloud-native applications has created the demand for scalable cloud storage infrastructure in the modern datacenter. Dell ECS is the third-generation object store platform from Dell Technologies. ECS is designed from the ground up to deliver modern cloud storage API, distributed data protection, and active/active availability spanning multiple data centers.

Elasticsearch® is a distributed, RESTful search and analytics engine capable of storing your data, and includes a smart solution to back up single indices or entire clusters to a remote shared filesystem, S3 or HDFS.

Explosive data growth in customers' Insight Engines, such as Elastic Stack, have resulted in the adoption of RESTful object storage for less frequently used, yet still searchable data, as a frozen tier. As an industry leader in object storage, ECS fulfills the requirements of an S3 compatible object platform for tiering Elastic data to that frozen tier. Additionally, Elastic's searchable snapshot feature allows for placement of the replica copy of hot data (shards) to be stored on ECS as a searchable snapshot, which reduces the replica storage costs for highly searched data.

This deployment guide describes how to configure the Elastic Stack components (Elasticsearch, Logstash and Kibana – or ELK) to leverage ECS as both the frozen tier and repository for Searchable Snapshots.

Elastic was populated with access log data from ECS though Syslog to demonstrate how a two-site ECS environment's data can be ingested, indexed by each VDC, searchable by Elastic, and displayed by means of dashboard widgets in Kibana to visualize the transactional data occurring in the ECS environment.

Revisions

Date	Description
July 2022	Initial release

Note: This document may contain language from third-party content that is not under Dell Technologies' control and is not consistent with current guidelines for Dell Technologies' own content. When such third-party content is updated by the relevant third parties, this document will be revised accordingly.

We value your feedback

Dell Technologies and the authors of this document welcome your feedback on this document. Contact the Dell Technologies team by email.

Authors: Christoffer Braendstrup, Jon Linder, and Rich Paulson

Note: For links to Dell ECS documentation, see the Dell Technologies ECS Info Hub.

Deployment Overview

After the necessary Elastic Stack components (Elasticsearch, Logstash, and Kibana – or ELK) are deployed, this section describes the steps to set up ECS as a frozen tier and as a repository for searchable snapshots. ECS access log data is pulled in through Syslog to populate the Elastic Stack and provides a quick method to validate the movement of data through Elastic's tiers of storage and searchability of snapshotted indices. Dashboard widgets in Kibana then display the ECS access log data.

Terminology

The following table provides definitions for some of the terms that are used in this document.

Table 1. Terminology

Term	Definition
ELK	"ELK" is the acronym for three open-source projects: Elasticsearch, Logstash, and Kibana.
Snapshot	The state of a system at a particular point in time.
Index management	Kibana's Index Management features are an easy, convenient way to manage your cluster's indices, data streams, and index templates.
ECS virtual data center (VDC)	A cluster, also generally referred to as a site or geographically distinct region, made up of a set of ECS infrastructure managed by a single fabric instance

Solution Architecture

The test environment deployed for ELK consisted of three Elasticsearch servers to provide quorum, a Logstash server, and a Kibana server. The ECS environment consisted of two federated ECS appliances.

Data was ingested into the ELK environment from ECS by the ECS syslog forwarder. ECS data access logs (which capture transactional data from client to ECS) were used for validating the frozen tier and for the searchable snapshot functions of Elastic. It also provides customers an overview of how you can ingest the ECS access log data and easily search and visualize the transactional activities of ECS.

Logical Component Layout

The following figure shows the layout of components used during the implementation.

ECS transactional logs from two ECS VDCs are shipped to Logstash, parsed, and sent to the Elasticsearch nodes for indexing. Two S3 snapshot repositories were created, one for scheduled snapshots and the other for snapshots that eventually get frozen. Finally, a lifecycle policy was set up to include two phases:

- **Hot Phase**: In which indexes are actively being queried and updated.
- **Frozen Phase**: In which the index is no longer being updated and is rarely queried. The information still needs to be searchable, but it is okay if those queries are slower.

Indexes in the hot phase were configured to roll more frequently to the next phase, which in our case is frozen. Note also from the following figure that replica shards are not required on the frozen tier because the object store is responsible for ensuring data durability.

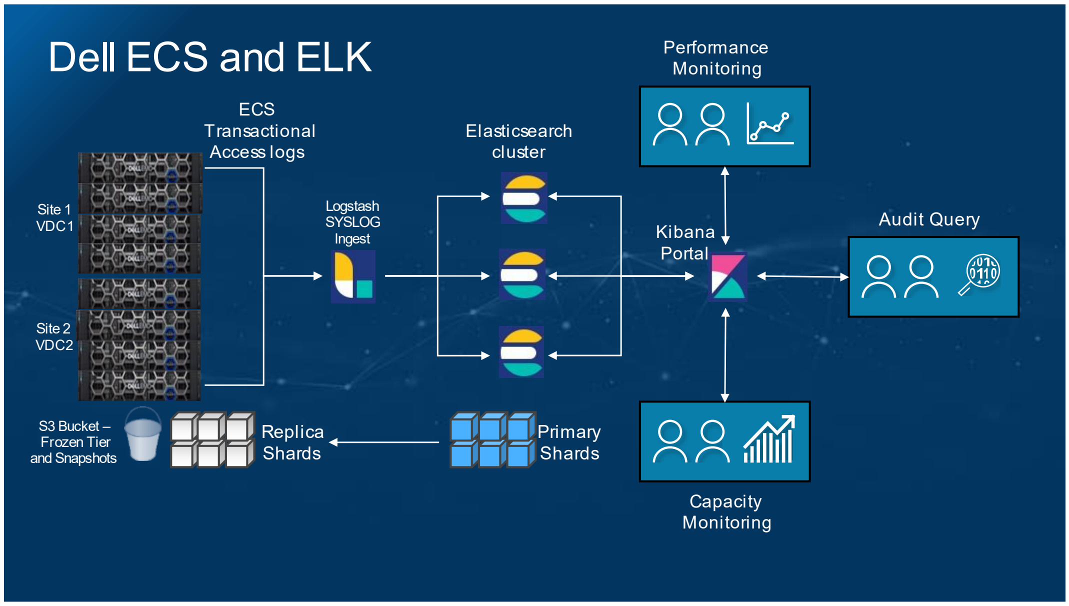


Figure 1. Logical Architectural Layout

Key Components

The following tables list the components deployed and their validated solution details.

Table 2. Dell Components

Component	Details
ECS Appliance	This solution has been validated with Dell ECS version 3.6 and above

Table 3. Elastic Stack Components

Component	Details
Elasticsearch	This solution has been validated with version 7.1.2 and above
Kibana	This solution has been validated with version 7.1.2 and above
Logstash	This solution has been validated with version 7.1.2 and above

Implementation

Deployment Steps

The following figure shows the high-level steps followed for deployment and validation on both ECS, and the ELK stack.

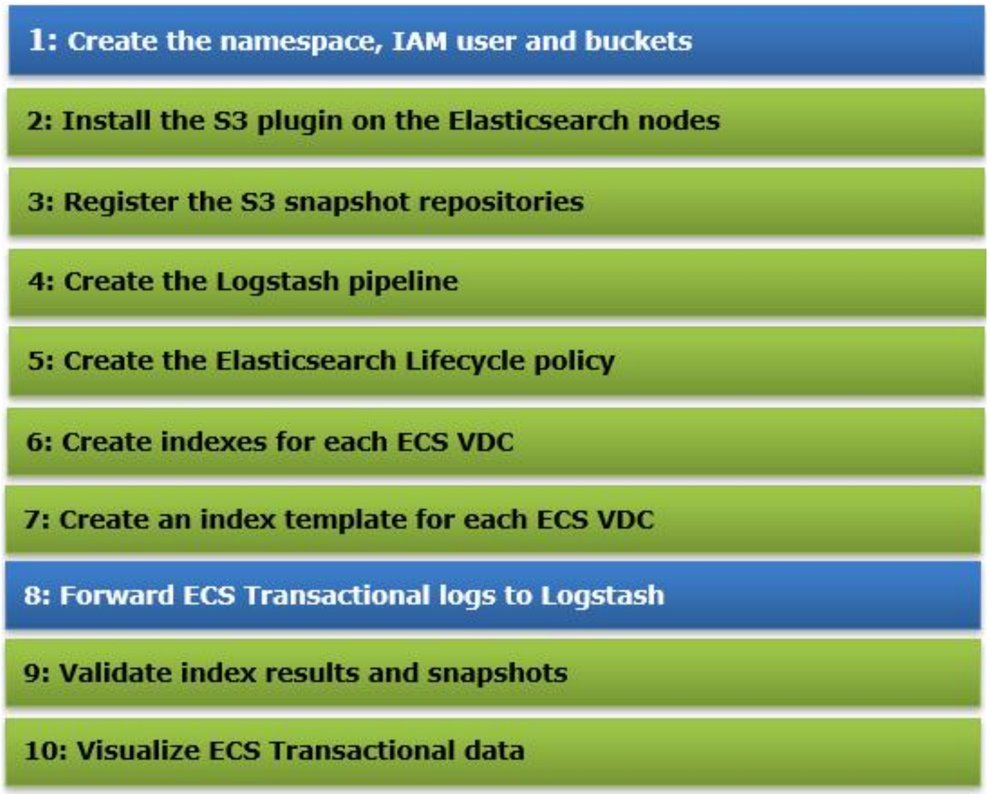


Figure 2. Deployment Steps on both ECS (shown in blue) and ELK (shown in green)

Note: Some of the following sections contain text that may not copy or paste correctly. Please be cautious when using these examples in your environment.

Note: Installing a trusted root certificate may be necessary if you are notified that the certificate of authority is not trusted on the host where the Elastic component is installed.

Deployment Steps

The following sections describe the pertinent deployment and configuration steps performed on the various ECS and Elastic components.

Create the IAM User and buckets

Two buckets were created in the ECS environment: one for the snapshot data and a second for the frozen data. An IAM (Identity Access Management) object user was created with Full Access to both buckets.

Create the ECS IAM User

ECS Identity and Access Management (IAM) enables you to have fine-grained and secure access to the ECS S3 resources. This functionality ensures that each access request to an ECS resource is identified, authenticated, and authorized.

An IAM user to be used for Elasticsearch S3 snapshots is created in the 'elasticsearch' namespace. Dell recommends a least permissions IAM Policy to use for the IAM user. See the Elastic site for more information about S3 permissions.

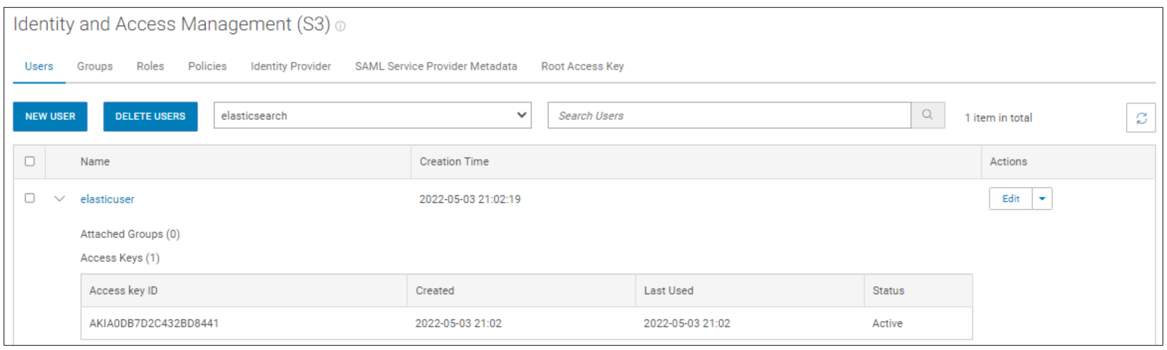


Figure 3. IAM User for Elasticsearch S3 snapshots

Figure 4. IAM User policy for Elasticsearch S3 snapshots

Create the ECS buckets for snapshot and frozen data

Buckets are containers for objects created in a namespace and are sometimes considered a logical container for sub-tenants. The snapshots will be uploaded to the buckets shown in the following figure..

ECS buckets can be created using the ECS Web Portal, REST API, or an S3 client such as S3browser or the AWS CLI.

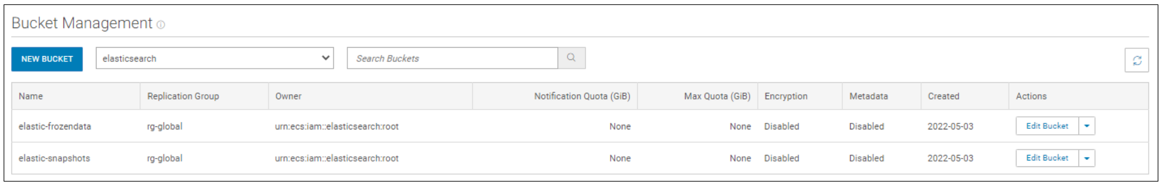


Figure 5. ECS buckets to store Elasticsearch snapshots

Install the S3 plugin on Elasticsearch

You must have the S3 Plugin installed on each Elasticsearch server. The commands used for this process are shown below for reference.

Note: The S3 plugin is included in Elasticsearch 8.x by default.

Reference the below link to install the S3 plugin for Elasticsearch 7.x

For earlier versions of Elasticsearch, install the S3 Plugin on each Elasticsearch node https://www.elastic.co/guide/en/elasticsearch/plugins/current/repository-s3.html.

When the S3 Plugin successfully completes and Elasticsearch services are restarted, you can use several methods to configure the S3 repository that points to ECS:

- Defined in the `elasticsearch.yml` file
- Created in the Snapshot and Restore section in Stack management
- Using the REST API

We chose to create the S3 repository using the Elasticsearch REST API. (Note that we used Elastic Stack version 8.1.3 in this document.)

Note: The API can be invoked using various tools such as curl or postman. You can also use the Developer Console from the Kibana UI.

Add the ECS Access and secret keys to the Elastic keystore

Most client settings can be added to the `elasticsearch.yml` configuration file except for secure settings, which you add to the Elasticsearch keystore. So, to keep our S3 credentials secure, we add them to the Elasticsearch keystore. The following commands prompt for the key:

```
./elasticsearch-keystore add s3.client.default.access_key
./elasticsearch-keystore add s3.client.default.secret_key
```

Note: Because keystore settings are secure and reloadable, we can add/update them without restarting the service or cluster by using the `reload_secure_settings` API.

Register a S3 snapshot repository

```
curl -X PUT "10.246.156.180:9200/_snapshot/ecs-repo" -H 'Content-Type:
application/json' -d'
{
  "type": "s3",
  "settings": {
    "bucket": "elastic-snapshots",
```

```
    "base_path": "snapshots",
    "endpoint": "s3.example.com",
    "protocol": "https",
    "path_style_access": "true"
  }
}
'
```

Register a S3 Repository for frozen snapshots

```
curl -X PUT "10.246.156.180:9200/_snapshot/ecs-frozen-repo" -H 'Content-
Type: application/json' -d'
{
  "type": "s3",
  "settings": {
    "bucket": "elastic-frozendata",
    "base_path": "snapshots",
    "endpoint": "s3.example.com",
    "protocol": "https",
    "path_style_access": "true"
  }
}
'
```

Note: if deployed in a corporate environment, there may be a need for corporate CA certificates to be added to the Elastic KeyStore for verification.

Let's look at each of the client settings in more detail. Note that the entire list of settings can be referenced here.

- **Bucket:** Name of the bucket to store our snapshots.
- **Base_path** (optional): Path to the repository data within the bucket.
- **Endpoint:** ECS requires an IP Load Balancer to distribute traffic to each node in the VDC.
- **Protocol:** Specifies whether HTTP or HTTPS is to be used.
- **Path_style_access:** Specifies whether to use Path or Virtual-Host style access.

There are a few methods to verify that the S3 snapshot repository was successful created. From Kibana, navigate to **Stack Management>Snapshot and Restore>Repositories**.

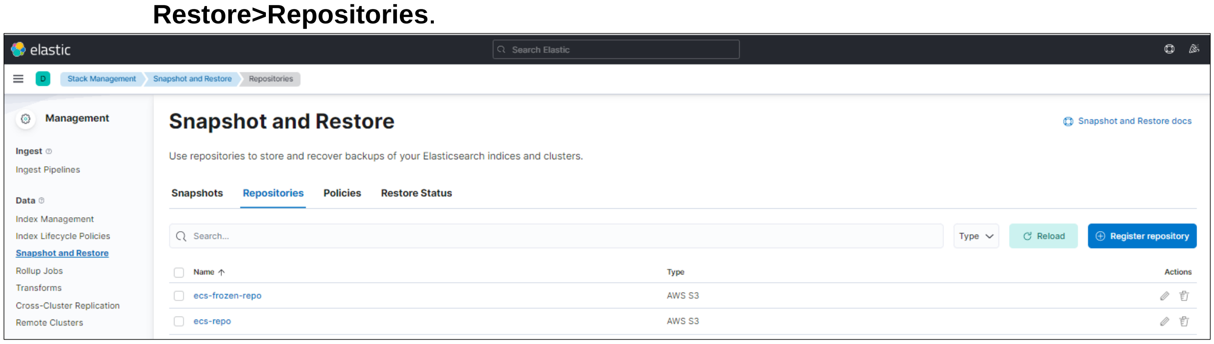


Figure 6. List the Snapshot Repositories

Or by using the `Get snapshot` API using curl or the developer console.

```
[root@logstash ~]# curl -X GET "10.246.156.180:9200/_snapshot?pretty"
{
  "ecs-repo" : {
    "type" : "s3",
    "uuid" : "WrFYcF-tQQWW7fHJ61rP7A",
    "settings" : {
      "bucket" : "elastic-snapshots",
      "path_style_access" : "true",
      "base_path" : "snapshots",
      "endpoint" : "s3.example.com",
      "protocol" : "http"
    }
  },
  "ecs-frozen-repo" : {
    "type" : "s3",
    "uuid" : "FH6JTt8qQHqcf174nxaICg",
    "settings" : {
      "bucket" : "elastic-frozendata",
      "path_style_access" : "true",
      "base_path" : "snapshots",
      "endpoint" : "s3.example.com",
      "protocol" : "http"
    }
  }
}
```

Verify the S3 snapshot repositories

Verify that the repositories are functional from the Kibana UI or using the REST API.

```
[root@es01 ~]# curl -X POST "localhost:9200/_snapshot/ecs-
repo/_verify?pretty"
{
  "nodes" : {
    "Q9xHsewaS2eWePOEJNQrTQ" : {
      "name" : "es01"
    },
    "OkKnge34S52qBhKdm84dgg" : {
      "name" : "es03"
    },
    "j-Er59lvTGCu0ia7OaU6DA" : {
      "name" : "es02"
    }
  }
}
```

```
[root@es01 ~]# curl -X POST "localhost:9200/_snapshot/ecs-frozen-
repo/_verify?pretty"
{
  "nodes" : {
    "Q9xHsewaS2eWePOEJNQrTQ" : {
      "name" : "es01"
    },
```

```
    "OkKnge34S52qBhKdm84dgg" : {
      "name" : "es03"
    },
    "j-Er59lvTGCu0ia7OaU6DA" : {
      "name" : "es02"
    }
  }
}
```

Create a Logstash pipeline

The data being ingested into Logstash from the two ECS VDCs is being tagged with a VDC indicator, depending on from which port (5005 or 5006) the syslog data was received. This allows an additional identifier filter to be put on incoming data streams so that customers can decide whether to perform searches across all VDC data, or data from a specific VDC.

The second portion of the Logstash pipeline is the GROK filter which accurately parses the access log data and assigns the data to the correct key / value pairs.

Certain fields in the access logs are either redundant or not needed and are subsequently removed in the mutate section.

The last portion of the pipeline shows how the output of the received data is sent to the appropriate VDC Index.

```
input {
        tcp {
            port => 5005
            type => syslog
            tags => ["VDC1"]
        }
             tcp {
                  port => 5006
                  type => syslog
                  tags => ["VDC2"]
             }
}

filter {
if "VDC1" in [tags] {
        grok {
                match => { "message" =>
["%{NOTSPACE:field1}%{MONTH:monthh}%{SPACE}+%{MONTHDAY:dayy}
%{TIME:timee}%{SPACE}%{IPORHOST:hostt}%{SPACE}%{WORD:wordd}%{SPACE}%{TIME
STAMP_ISO8601:ecstimestamp}%{SPACE}%{NOTSPACE:request_id}%{SPACE}%{IPORHO
ST:ecs_ip}:%{NUMBER:ecs_port}%{SPACE}%{IPORHOST:client_ip}:%{NUMBER:clien
t_port}%{SPACE}((?<user>[^-]{1}[^\s]*)|-)%{SPACE}((?<agent>[^-
]{1}[^\s]*)|-)%{SPACE}%{WORD:method}%{SPACE}((?<namespace>[^-
]{1}[^\s]*)|-)%{SPACE}((?<bucket>[^-]{1}[^\s]*)|-)%{SPACE}((?<key>[^-
]{1}[^\s]*)|-)%{SPACE}((?<query>[^-]{1}[^\s]*)|-
)%{SPACE}HTTP/%{NUMBER:http_version}%{SPACE}%{NUMBER:response_code}%{SPAC
E}%{NUMBER:duration}%{SPACE}%{NOTSPACE:upload_bytes}%{SPACE}%{NOTSPACE:do
wnload_bytes}%{SPACE}%{NUMBER:ecs_latency}%{SPACE}((?<range>[^-
]{1}[^\s]*)|-)%{SPACE}((?<copy>[^-]{1}[^\s]*)|-
```

```
)%{SPACE}%{NOTSPACE:deep_copy_size}%{SPACE}('X-Forwarded-For:
(%{IPORHOST:xforwardedfor}|-')| )"]
                        }
        }
        date {
                match => [ "ecstimestamp", "YYYY-MM-dd HH:mm:ss,SSS" ]
                timezone => "UTC"
                target => "@timestamp"
                }

        if [upload_bytes] == "-" {
                mutate {
                        replace => { "upload_bytes" => "0" }
                }
        }
        if [download_bytes] == "-" {
                mutate {
                        replace => { "download_bytes" => "0" }
                }
        }
        if [deep_copy_size] == "-" {
                mutate {
                        replace => { "deep_copy_size" => "0" }
                }
        }
        mutate {
                remove_field => ["field1", "monthh", "dayy", "timee",
"hostt", "wordd", "message", "type", "host", "port", "@version"]
                add_field => {"site" => "VDC1"}
                }
        }

if "VDC2" in [tags] {
        grok {
                match => { "message" =>
["%{NOTSPACE:field1}%{MONTH:monthh}%{SPACE}+%{MONTHDAY:dayy}
%{TIME:timee}%{SPACE}%{IPORHOST:hostt}%{SPACE}%{WORD:wordd}%{SPACE}%{TIME
STAMP_ISO8601:ecstimestamp}%{SPACE}%{NOTSPACE:request_id}%{SPACE}%{IPORHO
ST:ecs_ip}:%{NUMBER:ecs_port}%{SPACE}%{IPORHOST:client_ip}:%{NUMBER:clien
t_port}%{SPACE}((?<user>[^-]{1}[^\s]*)|-)%{SPACE}((?<agent>[^-
]{1}[^\s]*)|-)%{SPACE}%{WORD:method}%{SPACE}((?<namespace>[^-
]{1}[^\s]*)|-)%{SPACE}((?<bucket>[^-]{1}[^\s]*)|-)%{SPACE}((?<key>[^-
]{1}[^\s]*)|-)%{SPACE}((?<query>[^-]{1}[^\s]*)|-
)%{SPACE}HTTP/%{NUMBER:http_version}%{SPACE}%{NUMBER:response_code}%{SPAC
E}%{NUMBER:duration}%{SPACE}%{NOTSPACE:upload_bytes}%{SPACE}%{NOTSPACE:do
wnload_bytes}%{SPACE}%{NUMBER:ecs_latency}%{SPACE}((?<range>[^-
]{1}[^\s]*)|-)%{SPACE}((?<copy>[^-]{1}[^\s]*)|-
)%{SPACE}%{NOTSPACE:deep_copy_size}%{SPACE}('X-Forwarded-For:
(%{IPORHOST:xforwardedfor}|-')| )"]
                        }
        }
        date {
                match => [ "ecstimestamp", "YYYY-MM-dd HH:mm:ss,SSS" ]
                timezone => "UTC"
                target => "@timestamp"
                }
```

```
        if [upload_bytes] == "-" {
                mutate {
                        replace => { "upload_bytes" => "0" }
                }
        }
        if [download_bytes] == "-" {
                mutate {
                        replace => { "download_bytes" => "0" }
                }
        }
        if [deep_copy_size] == "-" {
                mutate {
                        replace => { "deep_copy_size" => "0" }
                }
        }
        mutate {
                remove_field => ["field1", "monthh", "dayy", "timee",
"hostt", "wordd", "message", "type", "host", "port", "@version"]
                add_field => {"site" => "VDC2"}
                }
        }

}

output {
       if "VDC1" in [tags] {
             elasticsearch {
                    index => "ecs-vdc1-index"
            hosts =>
["http://10.246.145.180:9200","http://10.246.156.181:9200","http://10.246
.156.182:9200"]
            }
       }
       if "VDC2" in [tags] {
        elasticsearch {
                    index => "ecs-vdc2-index"
            hosts =>
["http://10.246.156.180:9200","http://10.246.156.181:9200","http://10.246
.156.182:9200"]
            }
       }
 }
```

Our ECS VDC pipeline is now created at `/etc/logstash/conf.d/ecs-vdc-pipeline.conf` on the Logstash server.

We can see in the Logstash logs that our pipeline has been picked up as a source.

```
2022-05-19T19:27:11,107][INFO ][logstash.javapipeline][main]
Starting pipeline {:pipeline_id=>"main", "pipeline.workers"=>4,
"pipeline.batch.size"=>125, "pipeline.batch.delay"=>50,
"pipeline.max_inflight"=>500,
"pipeline.sources"=>["/etc/logstash/conf.d/ecs-vdc-
pipeline.conf"], :thread=>"#<Thread:0x6dcdcde1 run>"}
```

Create lifecycle policy

Only two-phase types were configured in the validation test, hot and frozen. Hot was configured to keep only one day of data to make transitions to frozen occur more quickly. Frozen was enabled and then configured to use the ECS S3 repository for both frozen data and searchable snapshots.

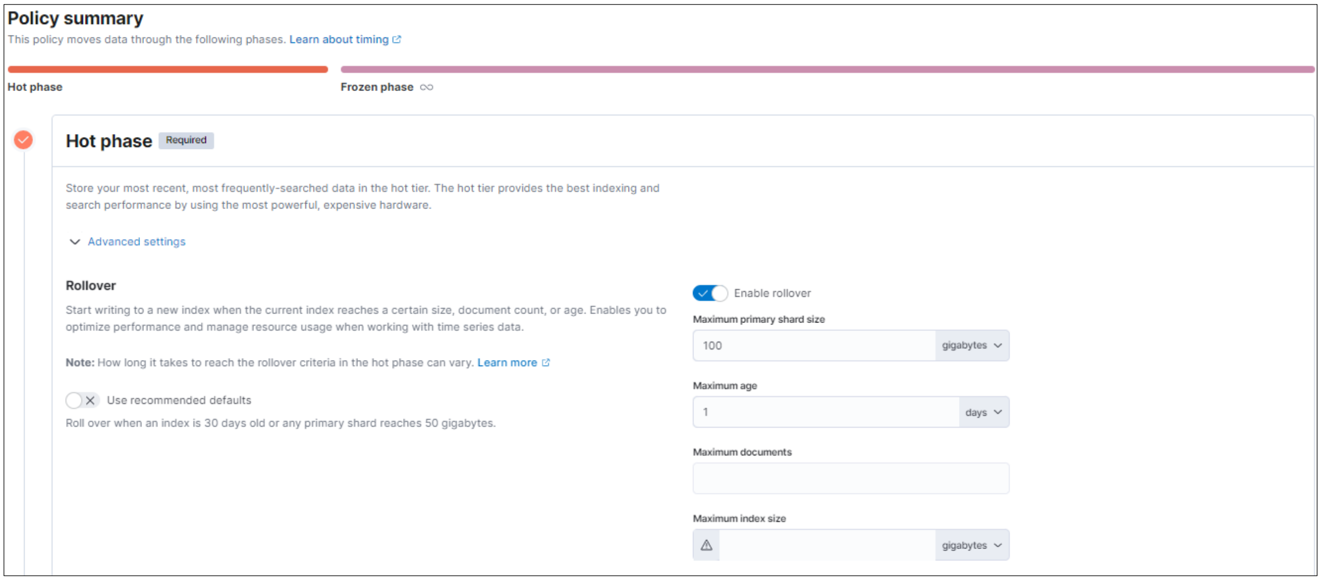


Figure 7. Hot phase lifecycle policy

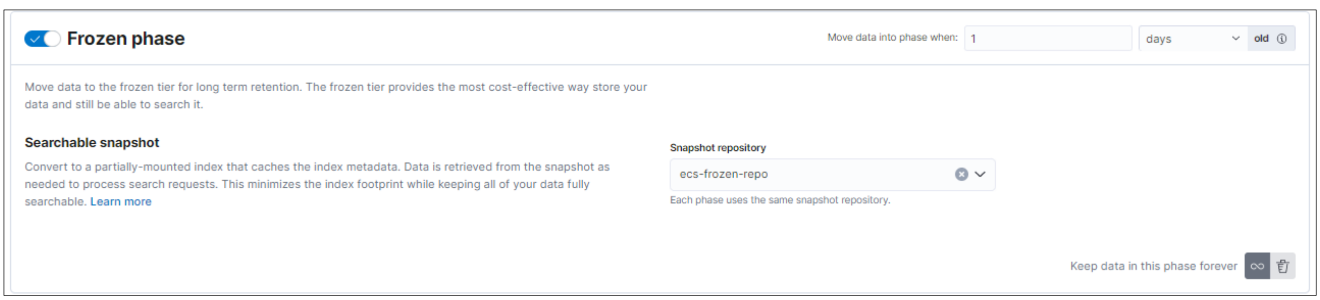


Figure 8. Frozen phase lifecycle policy

Create VDC Indexes

From the developer console in Kibana, PUT the first rollover index in, with its supported mapping.

VDC1

```
PUT <ecs-vdc1-index-{now}-000001>
{
  "mappings": {
      "properties": {
            "ecstimestamp": { "type": "date", "format": "yyyy-MM-
dd HH:mm:ss,SSS" },
            "request_id": { "type": "keyword" },
            "ecs_ip": { "type": "ip" },
            "ecs_port": { "type": "keyword" },
```

```
                "client_ip": { "type": "ip" },
                "client_port": { "type": "keyword" },
                "user": { "type": "keyword" },
                "agent": { "type": "text" },
                "method": { "type": "keyword" },
                "namespace": { "type": "keyword" },
                "bucket": { "type": "keyword" },
                "key": { "type": "text" },
                "query": { "type": "text" },
                "http_version": { "type": "float" },
                "response_code": { "type": "keyword" },
                "duration": { "type": "long" },
                "upload_bytes": { "type": "long" },
                "download_bytes": { "type": "long" },
                "ecs_latency": { "type": "long" },
                "range": { "type": "text" },
                "copy": { "type": "text" },
                "site": { "type": "keyword" },
                "deep_copy_size": { "type": "long" },
                "xforwardedfor": { "type": "ip" }
                }
        },
        "settings": {
                "index.lifecycle.name": "ecsfrozensearch",
                "index.lifecycle.rollover_alias": "ecs-vdc1-index",
                "number_of_shards": "3",
     "number_of_replicas": "1"
        },
        "aliases": {
                "ecs-vdc1-index":{
                        "is_write_index": true
                }
        }
}
```

VDC2

```
PUT <ecs-vdc2-index-{now}-000001>
{
  "mappings": {
       "properties": {
               "ecstimestamp": { "type": "date", "format": "yyyy-MM-dd
HH:mm:ss,SSS" },
               "request_id": { "type": "keyword" },
               "ecs_ip": { "type": "ip" },
               "ecs_port": { "type": "keyword" },
               "client_ip": { "type": "ip" },
               "client_port": { "type": "keyword" },
               "user": { "type": "keyword" },
               "agent": { "type": "text" },
```

```
                "method": { "type": "keyword" },
                "namespace": { "type": "keyword" },
                "bucket": { "type": "keyword" },
                "key": { "type": "text" },
                "query": { "type": "text" },
                "http_version": { "type": "float" },
                "response_code": { "type": "keyword" },
                "duration": { "type": "long" },
                "upload_bytes": { "type": "long" },
                "download_bytes": { "type": "long" },
                "ecs_latency": { "type": "long" },
                "range": { "type": "text" },
                "copy": { "type": "text" },
                "site": { "type": "keyword" },
                "deep_copy_size": { "type": "long" },
                "xforwardedfor": { "type": "ip" }
                }
        },
        "settings": {
                "index.lifecycle.name": "ecsfrozensearch",
                "index.lifecycle.rollover_alias": "ecs-vdc2-index",
                "number_of_shards": "3",
    "number_of_replicas": "1"
        },
        "aliases": {
                "ecs-vdc2-index":{
                        "is_write_index": true
                }
        }
}
```

Create index template

For both VDC indexes, an index template is then created. This ensures that as indexes ‘roll’ they can reference thistemplate for the new index to be created from.

VDC1

```
PUT /_index_template/template-ecs-vdc1-index
{
        "index_patterns": ["ecs-vdc1-index-*"],
        "template": {
                "mappings": {
                        "properties": {
                                "ecstimestamp": { "type": "date", "format":
"yyyy-MM-dd HH:mm:ss,SSS" },
                                "request_id": { "type": "keyword" },
                                "ecs_ip": { "type": "ip" },
                                "ecs_port": { "type": "keyword" },
                                "client_ip": { "type": "ip" },
                                "client_port": { "type": "keyword" },
                                "user": { "type": "keyword" },
                                "agent": { "type": "text" },
                                "method": { "type": "keyword" },
                                "namespace": { "type": "keyword" },
                                "bucket": { "type": "keyword" },
                                "key": { "type": "text" },
```

```
                            "query": { "type": "text" },
                            "http_version": { "type": "float" },
                            "response_code": { "type": "keyword" },
                            "duration": { "type": "long" },
                            "upload_bytes": { "type": "long" },
                            "download_bytes": { "type": "long" },
                            "ecs_latency": { "type": "long" },
                            "range": { "type": "text" },
                            "copy": { "type": "text" },
                            "site": { "type": "keyword" },
                            "deep_copy_size": { "type": "long" },
                            "xforwardedfor": { "type": "ip" }
                            }
              },
              "settings": {
                     "number_of_shards": 3,
                     "number_of_replicas": 1,
                     "index.lifecycle.name": "ecsfrozensearch",
                     "index.lifecycle.rollover_alias": "ecs-vdc1-index"
              }
       }
}
```

VDC2

```
PUT /_index_template/template-ecs-vdc2-index
{
       "index_patterns": ["ecs-vdc2-index-*"],
       "template": {
              "mappings": {
                     "properties": {
                            "ecstimestamp": { "type": "date", "format":
"yyyy-MM-dd HH:mm:ss,SSS" },
                            "request_id": { "type": "keyword" },
                            "ecs_ip": { "type": "ip" },
                            "ecs_port": { "type": "keyword" },
                            "client_ip": { "type": "ip" },
                            "client_port": { "type": "keyword" },
                            "user": { "type": "keyword" },
                            "agent": { "type": "text" },
                            "method": { "type": "keyword" },
                            "namespace": { "type": "keyword" },
                            "bucket": { "type": "keyword" },
                            "key": { "type": "text" },
                            "query": { "type": "text" },
                            "http_version": { "type": "float" },
                            "response_code": { "type": "keyword" },
                            "duration": { "type": "long" },
                            "upload_bytes": { "type": "long" },
                            "download_bytes": { "type": "long" },
                            "ecs_latency": { "type": "long" },
                            "range": { "type": "text" },
                            "copy": { "type": "text" },
                            "site": { "type": "keyword" },
```

```
                            "deep_copy_size": { "type": "long" },
                            "xforwardedfor": { "type": "ip" }
                            }
              },
              "settings": {
                     "number_of_shards": 3,
                     "number_of_replicas": 1,
                     "index.lifecycle.name": "ecsfrozensearch",
                     "index.lifecycle.rollover_alias": "ecs-vdc2-index"
              }
       }
}
```

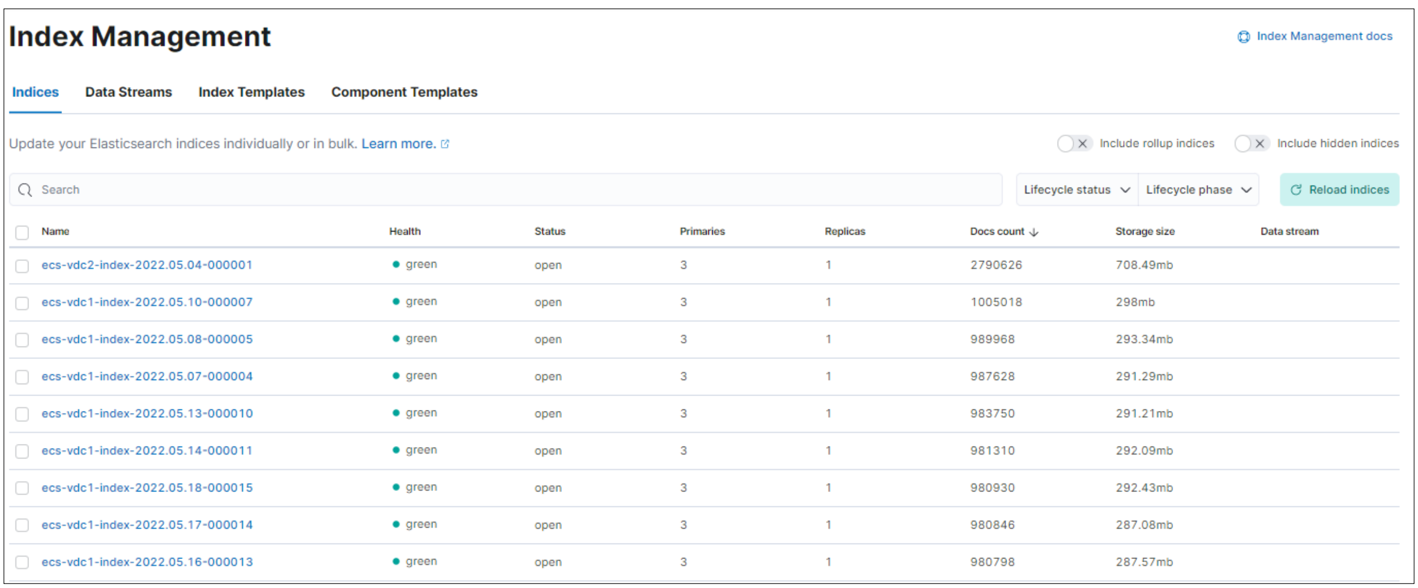
Forward ECS logs to Logstash

ECS can now be configured to forward the access logs for each node in a VDC to the Logstash server. Follow the steps outlined in the Knowledge Base article [ECS How to Export ECS access logs to external SYSLOG target].

Note: Make sure to configure VDC1 and VDC2 with the same ports that were defined in the Logstash pipeline configuration file.

Validate index results

Verify that indexes are being created and rolling over from the hot phase to frozen.



Name	Health	Status	Primaries	Replicas	Docs count	Storage size	Data stream
ecs-vdc2-index-2022.05.04-000001	green	open	3	1	2790626	708.49mb	
ecs-vdc1-index-2022.05.10-000007	green	open	3	1	1005018	298mb	
ecs-vdc1-index-2022.05.08-000005	green	open	3	1	989968	293.34mb	
ecs-vdc1-index-2022.05.07-000004	green	open	3	1	987628	291.29mb	
ecs-vdc1-index-2022.05.13-000010	green	open	3	1	983750	291.21mb	
ecs-vdc1-index-2022.05.14-000011	green	open	3	1	981310	292.09mb	
ecs-vdc1-index-2022.05.18-000015	green	open	3	1	980930	292.43mb	
ecs-vdc1-index-2022.05.17-000014	green	open	3	1	980846	287.08mb	
ecs-vdc1-index-2022.05.16-000013	green	open	3	1	980798	287.57mb	

Figure 9. ECS VDC indexes

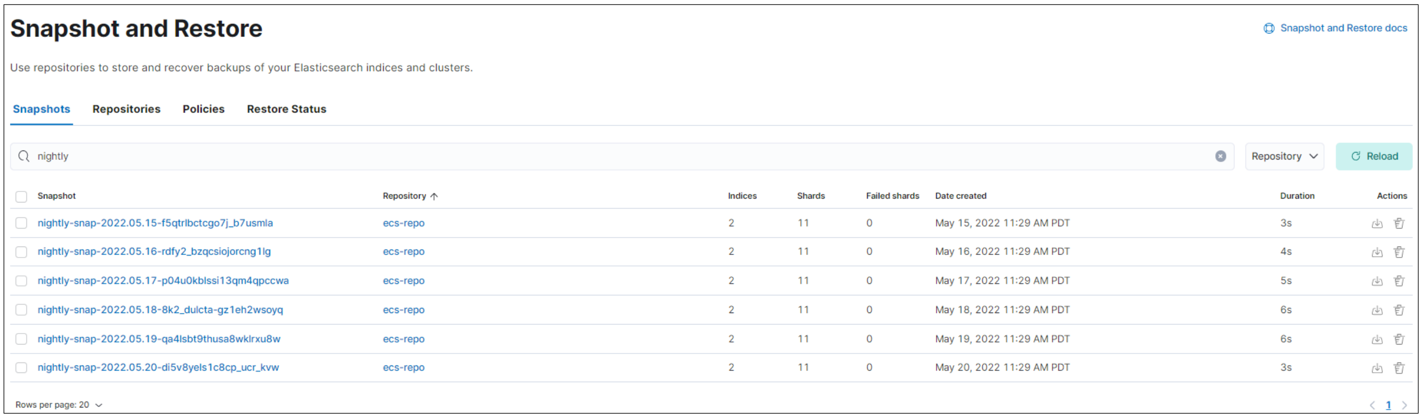


Figure 10. ECS VDC Snapshots

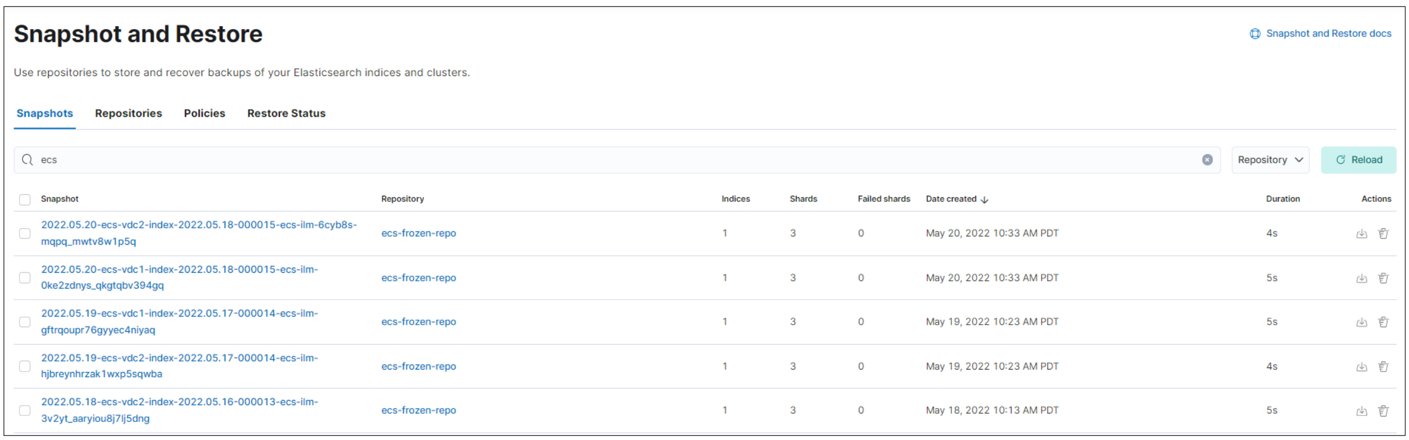


Figure 11. ECS VDC Frozen Snapshots

Searchable snapshots

Searchable snapshots let you use snapshots to search infrequently accessed and read-only data in a very cost-effective fashion. The frozen data tier uses searchable snapshots to reduce your storage and operating costs by storing the data on an object store such as Dell ECS.

Read more about searchable snapshots on the Elastic Site.

Visualize ECS transactional data

Kibana requires a data view to access the Elasticsearch data that you want to explore. A data view selects the data to use and allows you to define properties of the fields.

Note: Data views were previously named index patterns in earlier versions of Kibana.

Site specific data views

Create the data views for each ECS VDC.

Create the data view for VDC1 by navigating to **Stack management > Data Views > Create data view**.

In the Name field, enter ecs-vdc1-* and select @timestamp from the Timestamp field dropdown. Then click the ‘Create data view’ button.

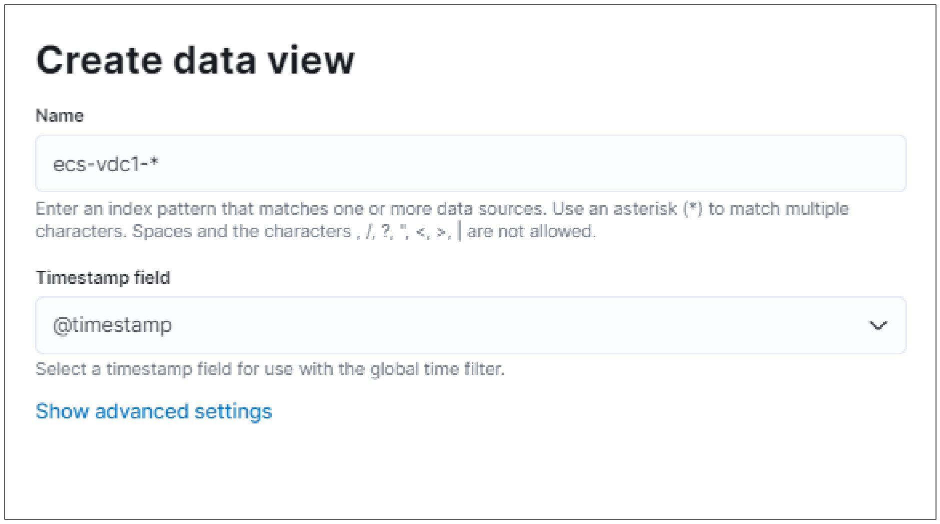


Figure 12. Create the data view for VDC1

Create the data view for VDC2 by navigating to **Stack management > Data Views > Create data view**.

In the Name field, enter ecs-vdc2-* and select @timestamp from the Timestamp field dropdown. Then click the ‘Create data view’ button.

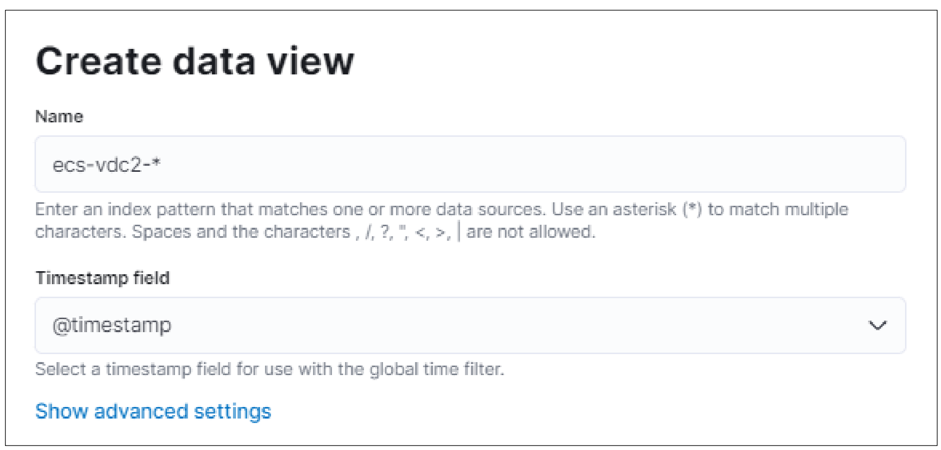


Figure 13. Create the data view for VDC2

Global Data Views

Create a data view that includes transactions from both VDCs.

Create a global data view to include both VDCs by navigating to **Stack management > Data Views > Create data view**.

In the Name field, enter ecs-* and select @timestamp from the Timestamp field dropdown. Then click the ‘Create data view’ button.

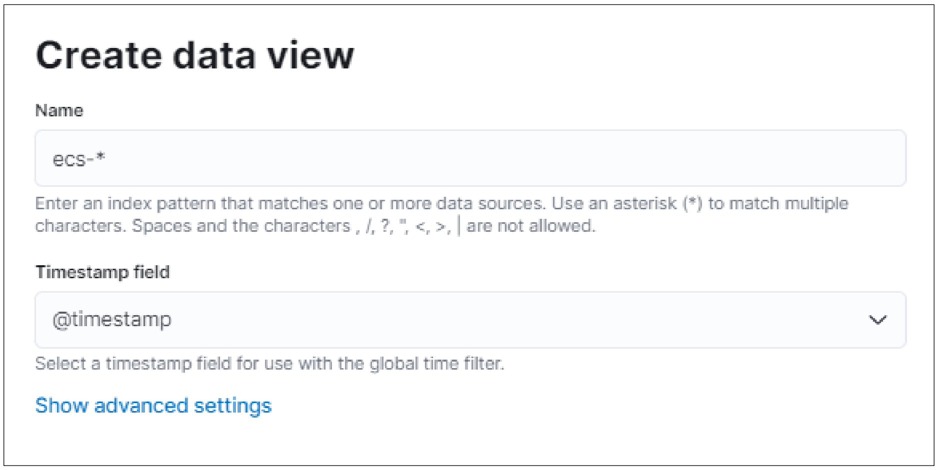


Figure 14. Create the global data view

Create Kibana dashboard

Data from the ECS access logs is now being ingested into the indices of Elasticsearch with the appropriate pattern to accurately parse and populate the fields in the index. To start visualizing data, select Dashboard from under the Analytics section.

Click **Create Dashboard**.

Three dashboards were created: one for Global ECS views, one for VDC1 specific views, and one for VDC2 specific views.

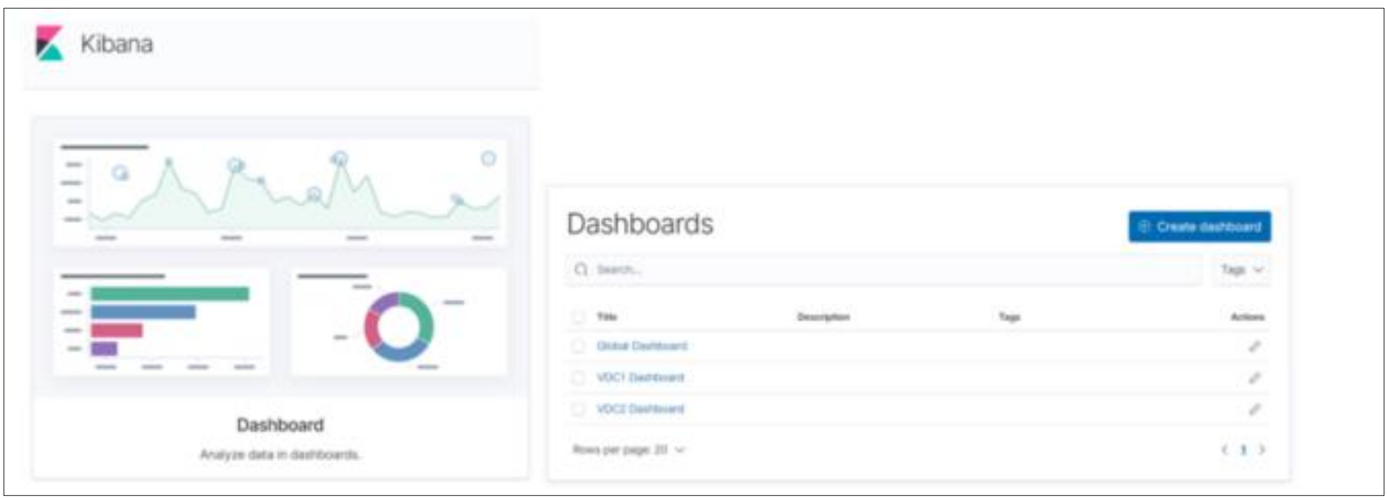


Figure 15. ECS dashboards

Looking at the Global Dashboard, we can see some examples of views aggregating values from both VDCs, such as Transaction Counts, Errors and Access Method Distributions.

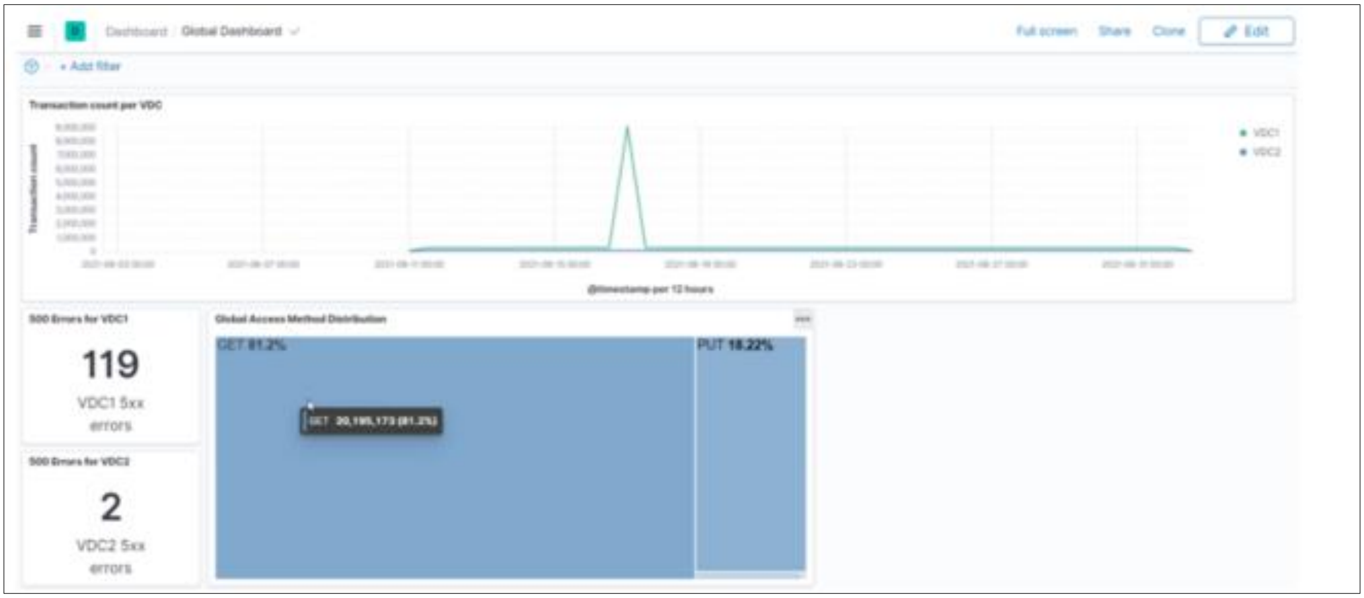


Figure 16. ECS Global Dashboard

Looking at the VDC specific dashboards, we can see some examples of views for data received from the ECS access logs, relating to transactions, throughput, access method, and Top N type reports.

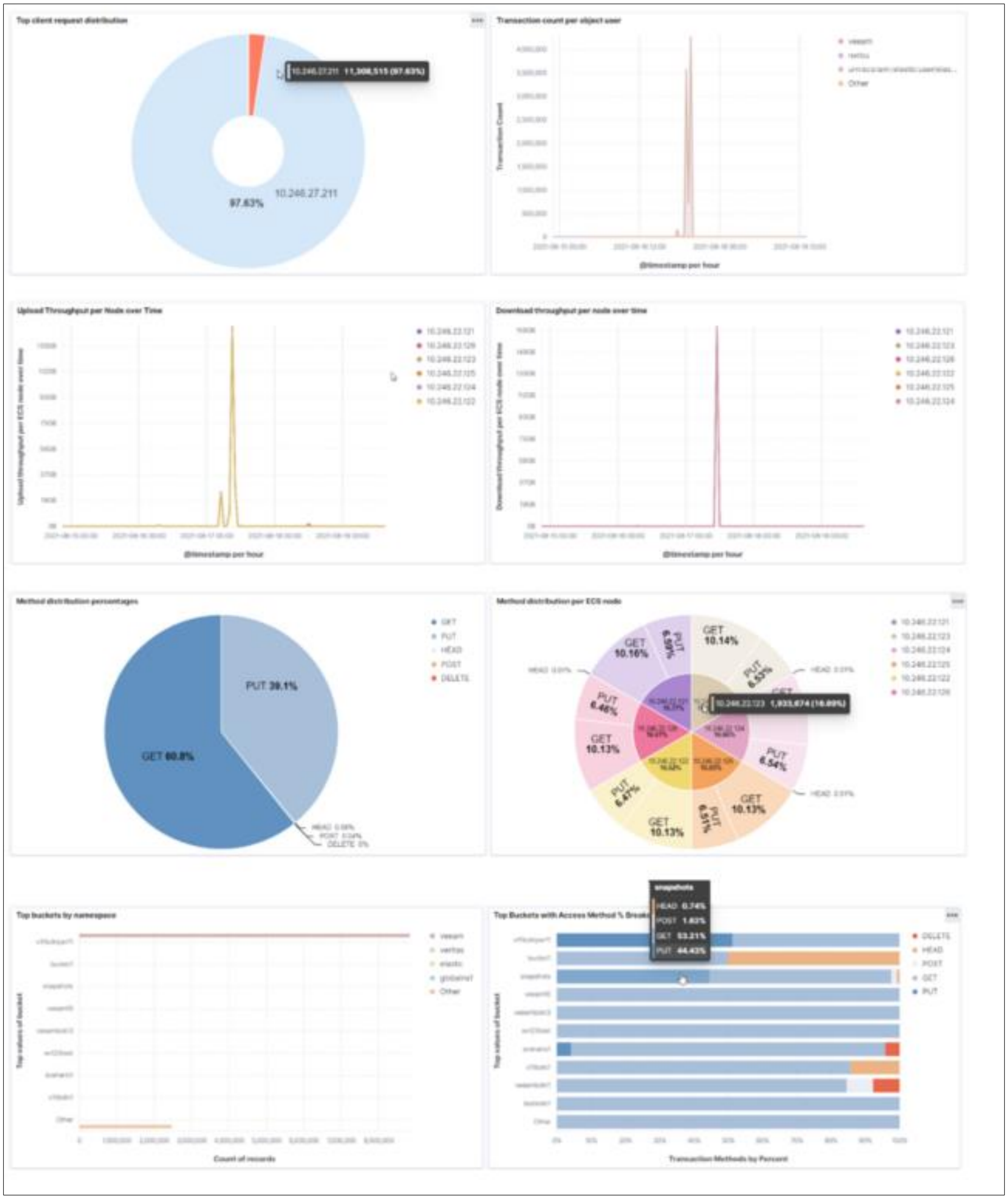


Figure 17. Example ECS site specific dashboard

Best practices

Dell Technologies recommends the following best practices when deploying Elasticsearch S3 snapshots.

Recommendation	Details
Managing traffic to ECS	The best practice to distribute S3 data traffic among the ECS nodes is to use an external IP load balancer.
Cleaning up multi-part uploads	Elasticsearch uses S3's multi-part upload process to upload larger blobs to the repository. If a multi-part upload cannot be completed, it must be aborted in order to delete any parts that were successfully uploaded. ECS supports S3 Lifecycle Configuration to automatically abort incomplete uploads when they reach a certain age. See the Data Access Guide for additional info.

References

Dell Technologies documentation

The following Dell Technologies documentation provides other information related to this document. Access to these documents depends on your login credentials. If you do not have access to a document, contact your Dell Technologies representative.

- *Dell ECS Info Hub*

Elastic documentation

The following Elasticsearch documentation provides information related to snapshot functionality.

- *Elasticsearch 7.x Snapshot and Restore*
- *Elasticsearch 8.x Snapshot and Restore*